

Guia Didático: O Laço FOR em Python

Explorando a versatilidade da iteração direta além da função range()

Conceito Fundamental: Em Python, o laço for é projetado para percorrer diretamente **objetos iteráveis** (coleções de dados como listas, textos, dicionários e tuplas). Ao contrário de outras linguagens, ele extrai elemento por elemento de forma automática, eliminando a necessidade de contadores manuais.

1. PERCORRENDO COLEÇÕES DE DADOS (LISTAS E TUPLAS)

Exemplo 1.1 — Iteração Direta Sobre uma Lista

O laço extrai cada elemento da lista sequencialmente e atribui à variável de controle.

```
produtos = ["Notebook", "Mouse", "Teclado"] for produto in produtos: print(f"Item em estoque: {produto}")
```

Como funciona:

- O Python identifica a lista produtos contendo 3 elementos.
- A variável produto recebe "Notebook", depois "Mouse" e por fim "Teclado".
- O laço encerra automaticamente assim que atinge o fim da coleção.

2. PROCESSAMENTO DE CADEIAS DE TEXTO (STRINGS)

Exemplo 2.1 — Leitura Caractere por Caractere

Strings são tratadas como sequências de caracteres, permitindo a varredura direta do texto.

```
palavra = "PYTHON" for letra in palavra: print(f"Letra processada: {letra}")
```

Como funciona:

- O texto "PYTHON" é desmembrado em seus 6 caracteres individuais.
- Muito útil para tarefas de validação de dados, buscas de padrões ou formatação.

3. ACOMPANHANDO POSIÇÃO E VALOR COM ENUMERATE()

Exemplo 3.1 — Acesso ao Índice Sem Contadores Manuais

A função enumerate() retorna o índice e o elemento correspondente a cada iteração.

```
linguagens = ["Python", "JavaScript", "C++"] for indice, linguagem in enumerate(linguagens): print(f"Posição {indice}: {linguagem}")
```

Como funciona:

- A cada volta, enumerate() gera um par de valores: a posição (0, 1, 2...) e o dado.
- Elimina a necessidade de criar e incrementar variáveis contadoras do tipo $i = i + 1$.

4. ITERAÇÃO SIMULTÂNEA EM MÚLTIPLAS COLEÇÕES COM ZIP()

Exemplo 4.1 — Percorrendo Duas Listas em Paralelo

A função `zip()` combina elementos de listas diferentes com base na mesma posição.

```
alunos = ["Ana", "Bruno", "Carla"] notas = [9.5, 8.0, 10.0] for nome, nota in zip(alunos, notas): print(f"O aluno(a) {nome} obteve a nota {nota}")
```

Como funciona:

- Na 1ª volta, agrupa o 1º aluno e a 1ª nota (Ana e 9.5).
- A repetição encerra assim que a menor das listas pare de fornecer elementos.

5. MAPEAMENTO DE DICIONÁRIOS ('DICT')

Exemplo 5.1 — Extraíndo Chaves e Valores com `.items()`

O método `.items()` desestrutura cada par chave-valor para processamento limpo.

```
cliente = {"nome": "Carlos", "idade": 34, "cidade": "São Paulo"} for chave, valor in cliente.items(): print(f"{chave.capitalize()}: {valor}")
```

Outras variações de busca em dicionários:

- `for chave in cliente.keys():` → Percorre apenas os nomes das chaves.
- `for valor in cliente.values():` → Percorre apenas os valores armazenados.

6. QUADRO RESUMO DOS RECURSOS DO FOR

Recurso / Estrutura	Objetivo Principal	Sintaxe de Exemplo
Listas / Tuplas	Iterar sobre sequências de elementos organizados.	<code>for item in lista:</code>
Strings ('str')	Processar texto caractere por caractere.	<code>for letra in texto:</code>
<code>`enumerate()`</code>	Capturar a posição (índice) e o valor juntos.	<code>for i, item in enumerate(lista):</code>
<code>`zip()`</code>	Percorrer duas ou mais listas em paralelo.	<code>for a, b in zip(lista1, lista2):</code>
<code>`dict.items()`</code>	Extrair os pares chave e valor de dicionários.	<code>for k, v in dados.items():</code>

